UNIVERSIDAD AUSTRAL

Trabajo Práctico de Procesamiento Avanzado de Imágenes

- Detección y clasificación de monedas y dados.
- Detección de patentes.

Integrantes:

- CASTILLO, Juan Ignacio
- FASOLATO, Matías
- TORREGIANI, Bautista

Profesor: Gonzalo Daniel Sad

ÍNDICE GENERAL

1. INTRODUCCIÓN 3

2. PROBLEMA 1 – Detección y clasificación de monedas y dados 4

2.1. Introducción de la imagen 4
2.2. Objetivo 5
2.3. Primeras pruebas de segmentación 5
2.4. Detección de bordes mediante gradiente 6
2.5. Umbralado de bordes 7
2.6. Construcción de la máscara inicial mediante morfología 8
2.7. Completar individualmente los objetos 9
2.8. Identificación y descripción geométrica de los objetos 10
2.9. Clasificación de las monedas por tamaño 11
2.10. Reconocimiento de los puntos por dados 13
2.11. Resultado final 14
2.12. Conclusión del Problema 1 15

3. PROBLEMA 2 – Detección automática de patentes 16

3.1. Introducción de las imágenes 16
3.2. Objetivo 16
3.3. Umbralado mediante Otsu 17
3.4. Identificación de los objetos mediante componentes conectadas 18
3.5. Filtrado de objetos por forma de letra 18
3.6. Agrupamiento de las letras alineadas 19
3.7. Selección del umbral de respaldo 20
3.8. Detección y recorte de la placa 21
3.9. Segmentación de los caracteres 21
3.10. Procesamiento de las 12 imágenes 22
3.11. Análisis de las patentes no detectadas 24
3.12. Conclusión del Problema 2 25

1- Introducción

En este trabajo práctico se resolvieron dos problemas mediante técnicas de procesamiento digital de imágenes implementadas en Python, utilizando las bibliotecas OpenCV, NumPy y Matplotlib.

El primero consistió en detectar y clasificar las monedas y los dados presentes en una fotografía de fondo no uniforme: segmentarlos de forma automática, clasificar las monedas según su tamaño y determinar el valor de la cara superior de cada dado.

El segundo problema consistió en detectar automáticamente la placa patente en imágenes de la vista anterior o posterior de distintos vehículos, segmentarla y luego segmentar los caracteres que la componen. En ambos casos el script debe informar y mostrar los resultados de cada una de las etapas de procesamiento.

Se emplearon técnicas de las unidades de la materia, en particular de transformación de escala y filtrado espacial, umbralado, detección de bordes y componentes conectadas, morfología matemática y descriptores con clasificación de objetos.

El informe se organiza en dos secciones, una por problema. En cada una se presenta la metodología, las pruebas que se descartaron y el motivo, los resultados obtenidos y las conclusiones.

2- Problema 1 – Detección y clasificación de monedas y dados

2.1. Introducción de la imagen

La imagen `monedas.jpg` es una fotografía a color de gran tamaño: 3871 píxeles de ancho por 2681 de alto, con 3 canales de color. En ella se observan monedas de distintos tamaños y dos dados, distribuidos sobre un fondo cuya intensidad no es uniforme: oscuro a la izquierda y cada vez más claro hacia la derecha.

Al cargarla con `cv2.imread()`, OpenCV entrega los tres canales en orden BGR. Por eso, para mostrarla correctamente con Matplotlib, que espera el orden RGB, se la convirtió con `cv2.cvtColor()` al formato RGB. Luego se la convirtió a escala de grises, pasando de tres canales a uno solo con la intensidad de cada píxel, y se analizó su histograma.

A partir de ese punto, casi todo el análisis se hizo sobre la versión en escala de grises: el estudio de la iluminación, el gradiente, el umbralado, la morfología y el reconocimiento de los puntos de los dados. La imagen RGB se utilizó solo para visualizar y para la imagen final con las etiquetas.

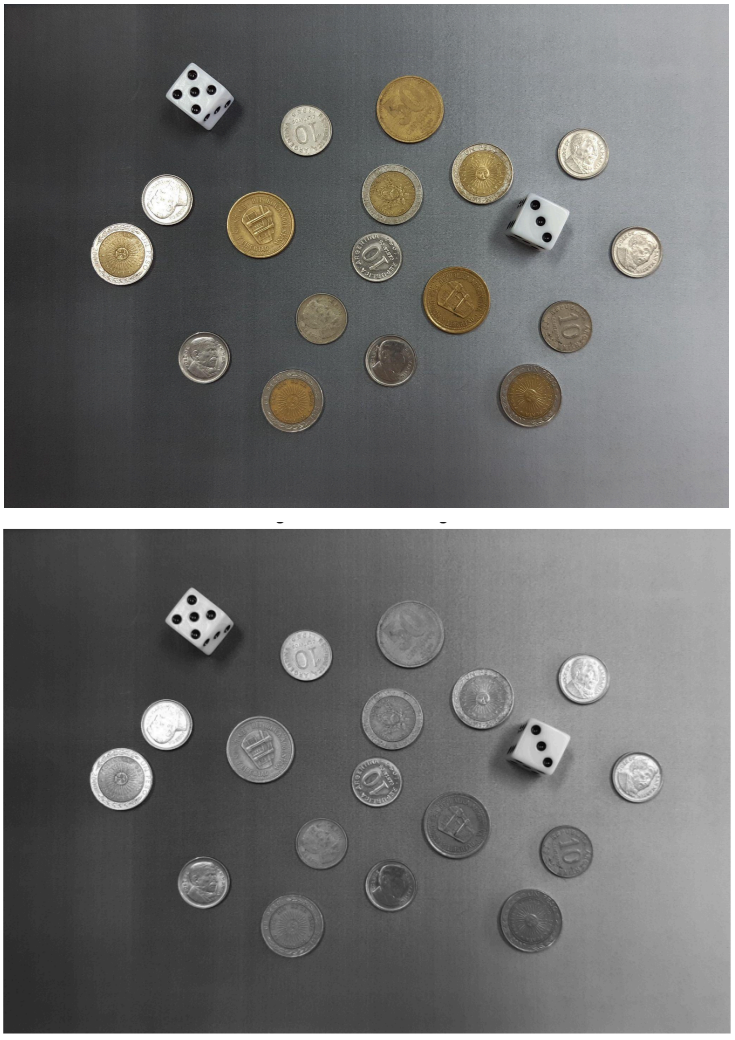


Figura 1. Imagen original e imagen en escala de grises utilizada para la segmentación de monedas y dados.

2.2. Objetivo

El objetivo fue desarrollar un algoritmo capaz de detectar automáticamente todos los objetos presentes en la imagen y clasificarlos. El procedimiento debía resolver tres tareas:

- **Segmentación:** separar las monedas y los dados del fondo, obteniendo una región independiente para cada objeto.
- **Clasificación de monedas:** distinguir los diferentes tipos de monedas a partir de sus características geométricas y contabilizar la cantidad de cada categoría.
- **Reconocimiento de dados:** detectar los puntos de la cara superior de cada dado, determinar el valor representado y contabilizar los resultados.

Para lograrlo se combinaron técnicas de reducción de escala, suavizado, detección de bordes, umbralado, operaciones morfológicas, componentes conectadas y descriptores geométricos.

2.3. Primeras pruebas de segmentación

Una primera aproximación consistió en convertir la imagen a escala de grises, analizar su histograma y aplicar un umbralado global mediante el método de Otsu, que calcula el umbral automáticamente a partir de la distribución de intensidades. El umbral obtenido fue T = 115. El resultado no separó los objetos del fondo: dividió la imagen según la iluminación. En la zona oscura las monedas y el dado quedaron blancos sobre negro, pero en la zona clara el fondo también pasó a blanco y los objetos se confundían con él.



Figura 2. Resultado de la primera prueba de segmentación mediante el método de Otsu.

Posteriormente se probó el umbralado adaptativo, tanto mediante el cálculo de la media local como mediante el método gaussiano. Esta alternativa produjo resultados visualmente más favorables, ya que permitió conservar una mayor cantidad de información de los objetos al calcular umbrales diferentes según el entorno de cada píxel.

Sin embargo, al analizar la máscara obtenida mediante componentes conectados, se detectaron 4093 regiones independientes. Esto evidenció una importante fragmentación de la imagen, con numerosas pequeñas regiones blancas que impedían identificar correctamente cada objeto.

Se intentó mejorar el resultado aplicando un filtro gaussiano previo y operaciones morfológicas de apertura y cierre. Sin embargo, esta combinación tampoco permitió obtener una segmentación adecuada: la cantidad de componentes llegó a aumentar hasta 15960. Por lo tanto, se descartó esta estrategia como procedimiento definitivo.

Estas pruebas mostraron que no era suficiente con obtener una imagen binaria visualmente más completa: hacía falta conseguir contornos definidos y cerrados para identificar cada objeto como una única región. Además, los cambios bruscos de intensidad en el borde de cada objeto se mantienen a ambos lados de la imagen aunque el brillo del fondo cambie, por lo que se optó por una estrategia basada en el gradiente.

2.4. Detección de bordes mediante gradiente

A partir de los inconvenientes encontrados con el umbralado directo, se optó por una estrategia basada en la detección de bordes.

En primer lugar, se redujo la escala de la imagen al 25 % de sus dimensiones originales mediante `cv2.resize()`, utilizando la interpolación `INTER_AREA`. Esta operación permitió trabajar con una representación más pequeña, reduciendo el detalle fino y el costo de procesamiento. Luego se aplicó un suavizado gaussiano con una ventana de 5 × 5 píxeles, con el objetivo de reducir pequeñas variaciones de intensidad antes de calcular el gradiente.

Para detectar los cambios de intensidad se utilizaron los operadores Sobel en las direcciones horizontal y vertical, y a partir de ambas derivadas se calculó la magnitud del gradiente. Los valores altos corresponden a los bordes de monedas y dados. La magnitud se normalizó al rango de 0 a 255 para visualizarla y umbralarla.

En la magnitud obtenida el fondo queda casi negro y parejo de izquierda a derecha, es decir que la variación de iluminación desaparece, y los contornos de los 19 objetos se ven en ambos lados de la imagen.

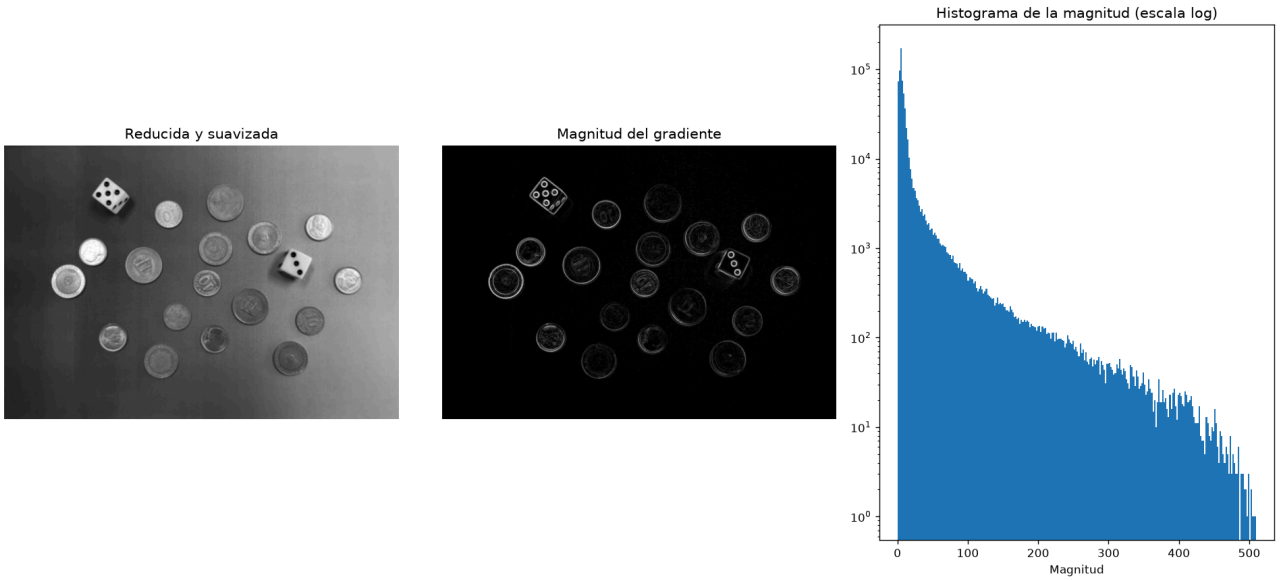


Figura 3. Imagen reducida y suavizada, magnitud del gradiente e histograma de magnitudes.

2.5. Umbralado de los bordes

Una vez obtenida la magnitud del gradiente, fue necesario convertirla en una representación binaria para identificar los bordes más relevantes.

Inicialmente se calculó un umbral de referencia mediante Otsu aplicado sobre la magnitud del gradiente. Además, se realizaron pruebas con tres umbrales fijos: 10, 20 y 30.

Luego de comparar visualmente los resultados, se seleccionó el umbral 20, que permitió conservar una cantidad adecuada de bordes para continuar con el procesamiento.

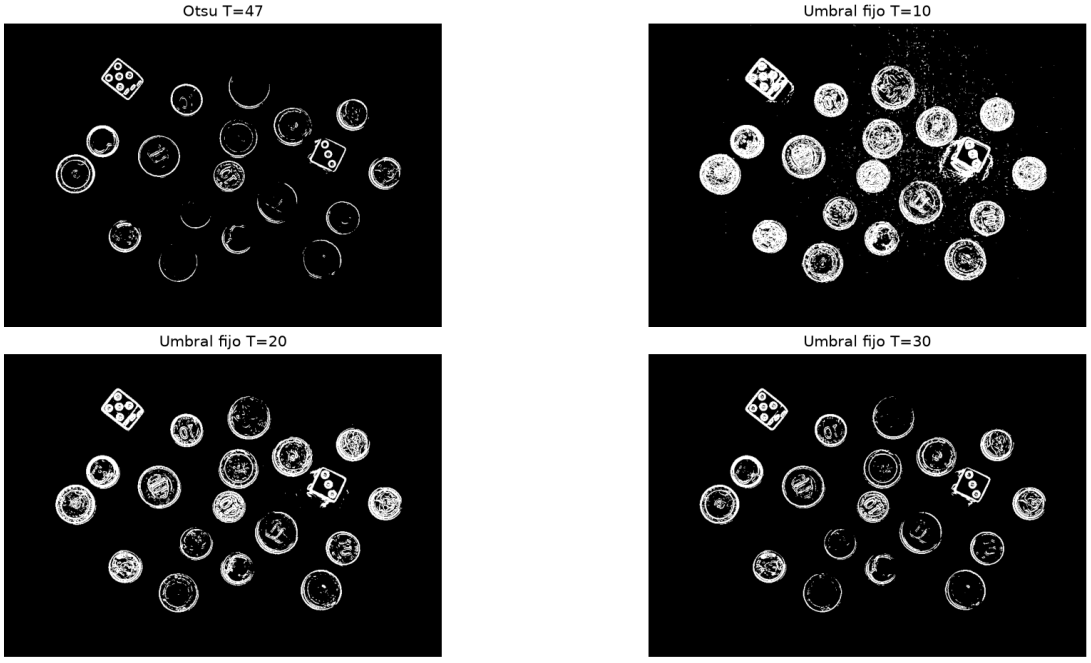


Figura 4. Comparación de los resultados obtenidos mediante Otsu y los umbrales fijos 10, 20 y 30.

Umbral	Resultado observado
Otsu (T = 47)	Fondo limpio, pero los contornos se cortan y varias monedas quedan como arcos sueltos
10	Contornos gruesos y completos, pero con mucho grano en el centro y la derecha
20	Fondo limpio y casi todos los contornos continuos (elegido)
30	Fondo limpio, pero los contornos se cortan en muchos tramos

Se seleccionó el umbral 20 por ser el mejor equilibrio entre fondo limpio y contornos continuos.

Luego se aplicó un filtrado por componentes conectados para eliminar regiones aisladas que podrían corresponder a ruido: se conservaron solo las componentes de área igual o superior a 50 píxeles.

Esta etapa produjo una imagen de bordes más limpia, que se utilizó como entrada para construir las máscaras de los objetos.

2.6. Construcción de la máscara inicial mediante morfología

El principal inconveniente de trabajar directamente con bordes es que estos no necesariamente forman contornos completamente cerrados. Cuando existen pequeñas interrupciones, no es posible identificar correctamente el interior de cada objeto.

Para resolverlo se utilizaron operaciones morfológicas de dilatación y relleno de regiones:

1. **Dilatación** de los bordes limpios con un elemento estructurante elíptico, que conecta las pequeñas discontinuidades.
2. **Relleno de huecos internos:** se identificaron las regiones del fondo mediante componentes conectados con conectividad 4. La región conectada con la esquina de la imagen se consideró fondo externo, y las regiones encerradas por los contornos se incorporaron a la máscara del objeto.
3. **Erosión** con el mismo elemento estructurante, para compensar la expansión producida por la dilatación.

Se compararon dos tamaños de elemento estructurante: 7 × 7 y 11 × 11. El de mayor tamaño provocaba que algunos objetos cercanos terminaran uniéndose, por lo que se seleccionó el de 7 × 7.

Previamente se había probado rellenar directamente los contornos externos de los bordes dilatados. Esa variante funcionaba donde el contorno estaba cerrado, pero dejaba como anillos vacíos a las monedas cuyo contorno tenía un corte. Con elementos más grandes (15 × 15 y 25 × 25) se cerraban más contornos, pero también se pegaban monedas vecinas. Por eso se pasó al relleno desde el fondo, con un elemento chico.

Con la máscara inicial casi todos los objetos quedan completos, pero algunos siguen incompletos: la moneda grande de la parte superior, tres monedas chicas del centro y el dado de la derecha tienen huecos o un contorno abierto. Esto se resuelve en la etapa siguiente.

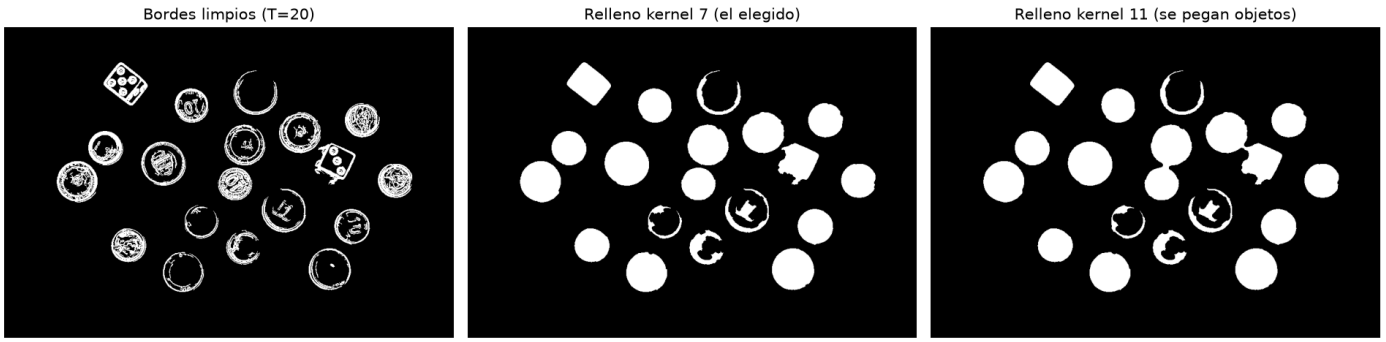


Figura 5. Comparación entre los bordes limpios y las máscaras obtenidas con elementos estructurantes de 7 × 7 y 11 × 11.

2.7. Completar individualmente los objetos

A pesar de contar con una máscara inicial, todavía era necesario mejorar el cierre de algunos contornos para que cada moneda y cada dado quedaran representados por una única región.

Una operación de cierre grande aplicada sobre toda la máscara no sirve: el elemento que cierra el hueco de una moneda también pega a la moneda vecina. Por eso se identificaron las componentes conectadas de la máscara y se procesó cada una de manera independiente, trabajando sobre su rectángulo delimitador.

De esta forma el elemento estructurante no puede alcanzar a los demás objetos. Sobre cada componente se aplicó un cierre morfológico, seguido nuevamente del relleno de huecos.

Se compararon dos tamaños de kernel: 81 × 81 y 111 × 111. El objetivo era conseguir que los contornos quedaran suficientemente cerrados, sin perder la separación entre los distintos objetos.

Finalmente se seleccionó el kernel de 111 × 111 para construir la máscara definitiva.

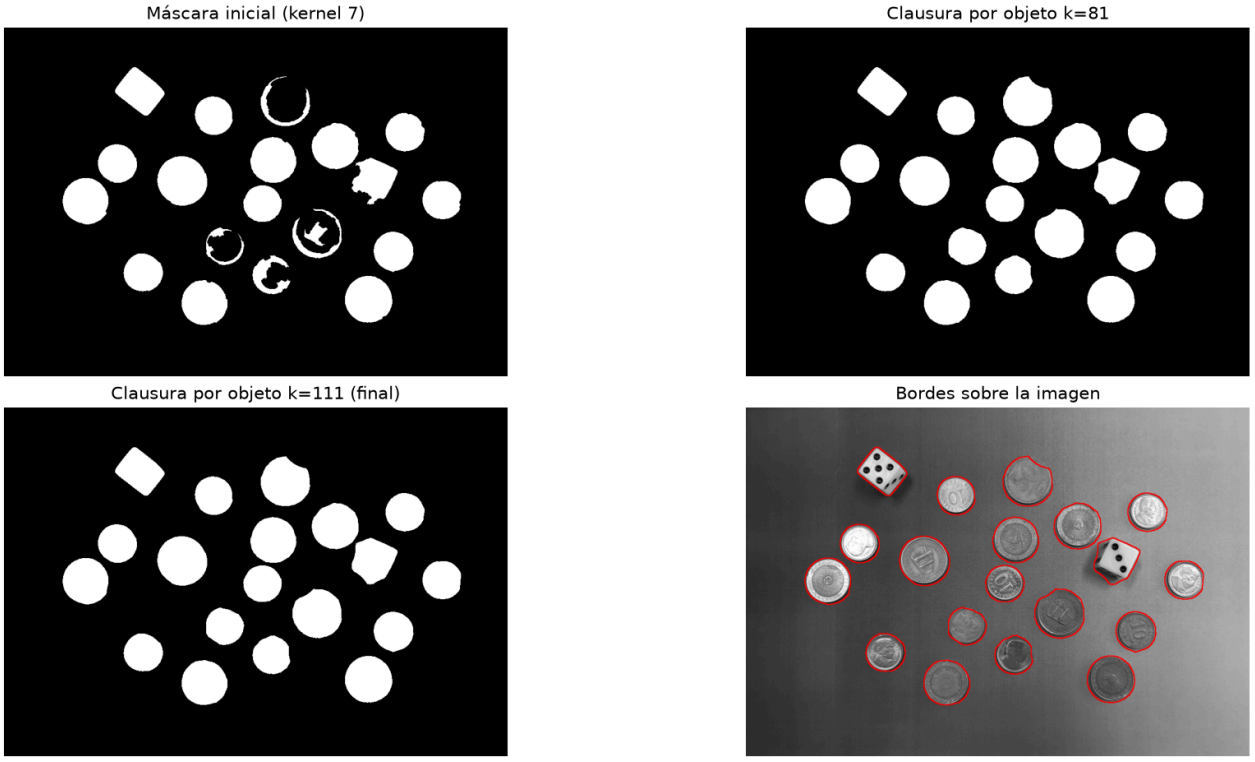


Figura 6. Comparación de las máscaras obtenidas con el cierre individual de los objetos (kernels de 81 × 81 y 111 × 111) y superposición de los contornos finales sobre la imagen original.

Esta etapa resultó fundamental, ya que permitió pasar de una representación basada en bordes a una máscara formada por regiones completas, adecuada para realizar el análisis geométrico posterior.

2.8. Identificación y descripción geométrica de los objetos

Una vez obtenida la máscara definitiva, se utilizó cv2.connectedComponentsWithStats() para identificar las regiones independientes presentes en ella.

Para cada componente se calcularon diferentes características:

- Área: cantidad de píxeles que forman el objeto.
- Perímetro: longitud del contorno externo, obtenida mediante cv2.arcLength().
- Centroide: posición central de la región.
- Circularidad: descriptor que permite evaluar cuánto se aproxima la forma del objeto a una circunferencia.

La circularidad se calculó a partir del área A y del perímetro P:

$$C = \frac{4\pi A}{P^2}$$

Vale 1 para un círculo perfecto y disminuye en formas con esquinas, como un cuadrado.

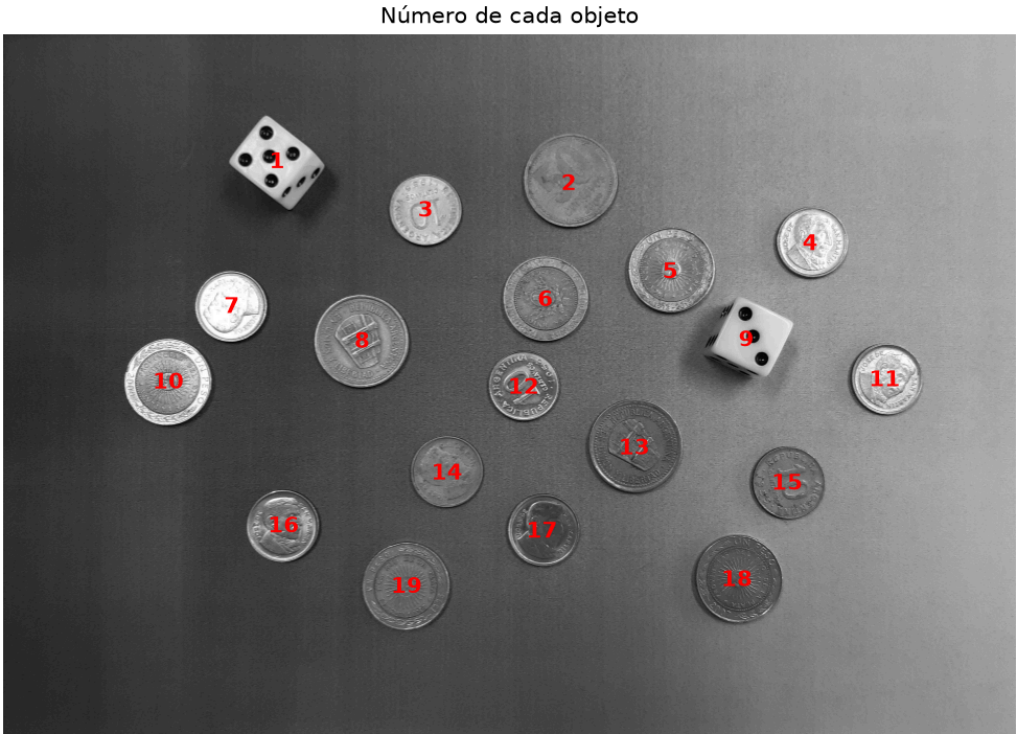


Figura 7. Identificación de los componentes conectados y numeración de los objetos detectados.

2.9. Clasificación de monedas y dados

A partir de los descriptores geométricos calculados se realizó una primera clasificación para distinguir monedas de dados.

Se utilizó la circularidad como criterio principal, estableciendo un umbral de 0,87 (valor intermedio que se da entre el salto de ambos grupos):

- Los objetos con circularidad **inferior a 0,87** se clasificaron como **dados**.
- Los objetos con circularidad **igual o superior a 0,87** se clasificaron como **monedas**.

Esta distinción se basa en que las monedas presentan una geometría aproximadamente circular, mientras que los dados poseen una forma más cercana a la cuadrada.

Una vez identificadas las monedas, se utilizó el algoritmo K-means para agruparlas según su área. Se fijó previamente k = 3, ya que se buscaba distinguir tres categorías de tamaño.

Los centros de los grupos se ordenaron de menor a mayor para asignar las etiquetas de moneda chica, mediana y grande, de modo que cada moneda quedó clasificada automáticamente según el grupo al que pertenecía.

Los grupos resultaron bien separados: las áreas dentro de cada tipo son parecidas y entre tipos hay saltos grandes, por lo que el área sola alcanza como descriptor.

Grupo	Monedas	Área aproximada (px)	Denominación asignada
Chica	9	4.100 – 4.600	$0,10
Mediana	5	6.000 – 6.500	$1
Grande	3	7.000 – 7.200	$0,50

La denominación se asignó por tamaño relativo: la moneda más chica corresponde a $0,10, la mediana a $1 y la más grande a $0,50.

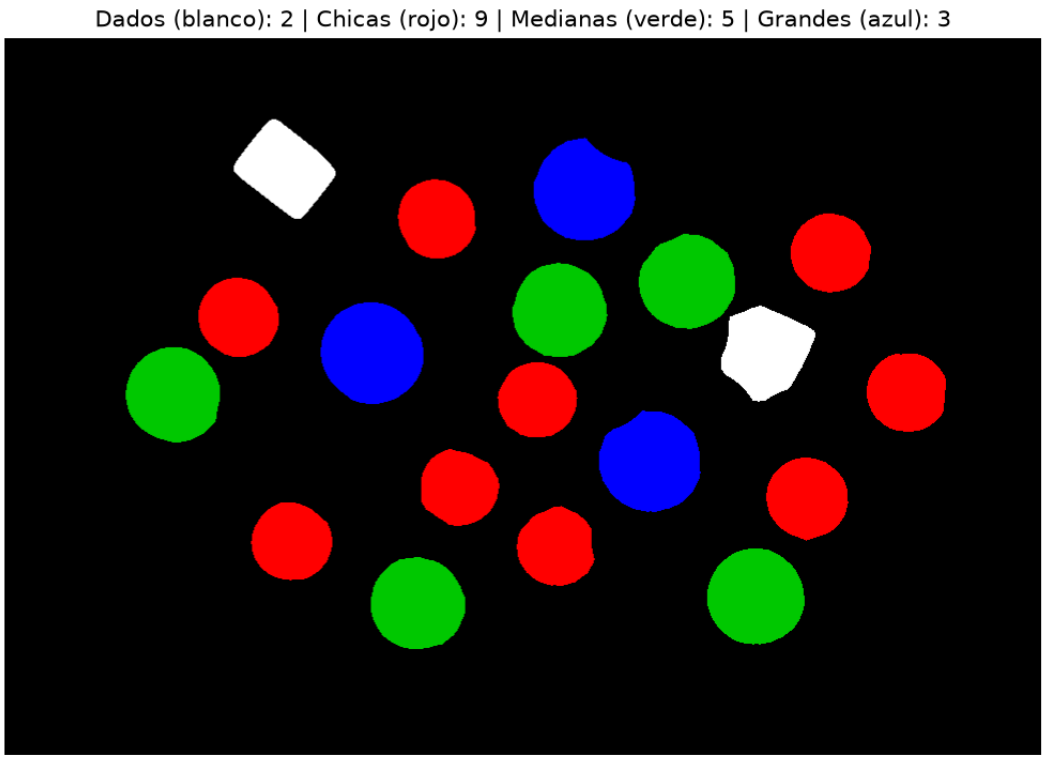


Figura 8. Clasificación de los dados y agrupamiento de las monedas mediante K-means, utilizando un color diferente para cada categoría.

2.10 Reconocimiento de los puntos de los dados

Para determinar el valor representado en la cara superior de cada dado se trabajó individualmente sobre cada uno de los objetos previamente identificados.

En primer lugar, se recuperaron los recortes correspondientes a los dados a partir de la imagen original en escala de grises, utilizando las coordenadas obtenidas durante la segmentación. Esto permitió conservar una mayor resolución para el reconocimiento de sus puntos.

Posteriormente se trasladó la máscara del dado al tamaño del recorte original y se aplicó una erosión con un elemento estructurante elíptico de 25 × 25 píxeles. El objetivo fue excluir los bordes del dado y concentrar el análisis en su región interior, evitando confundir los límites del objeto con los puntos de su cara superior.

Sobre los píxeles interiores se calculó un umbral automático mediante Otsu. Luego se seleccionaron los píxeles de intensidad inferior a dicho umbral, ya que los puntos de los dados presentan una tonalidad más oscura que la superficie que los rodea.

La imagen resultante se analizó mediante componentes conectados. Para distinguir los puntos reales de pequeñas imperfecciones, se establecieron dos criterios:

- Área mínima de 1000 píxeles.
- Circularidad mínima de 0,7.

Únicamente las componentes que cumplieron ambas condiciones fueron consideradas puntos válidos. Finalmente, el valor asignado a cada dado se obtuvo contando la cantidad de componentes que superaron estos filtros.

Se necesitan los dos criterios porque las componentes oscuras que aparecen dentro del dado no son todas puntos de la cara superior:

Componente	Área (px)	Circularidad	¿Se cuenta?
Punto de la cara superior	2.050 – 2.350	0,86 o más	Sí
Punto de una cara lateral (se ve en escorzo)	alrededor de 730	alrededor de 0,4	No, por ambos criterios
Mancha oscura grande en el dado de la derecha (probablemente su sombra)	19.442	0,2	No, por circularidad
Ruido	1 – 2	–	No, por área

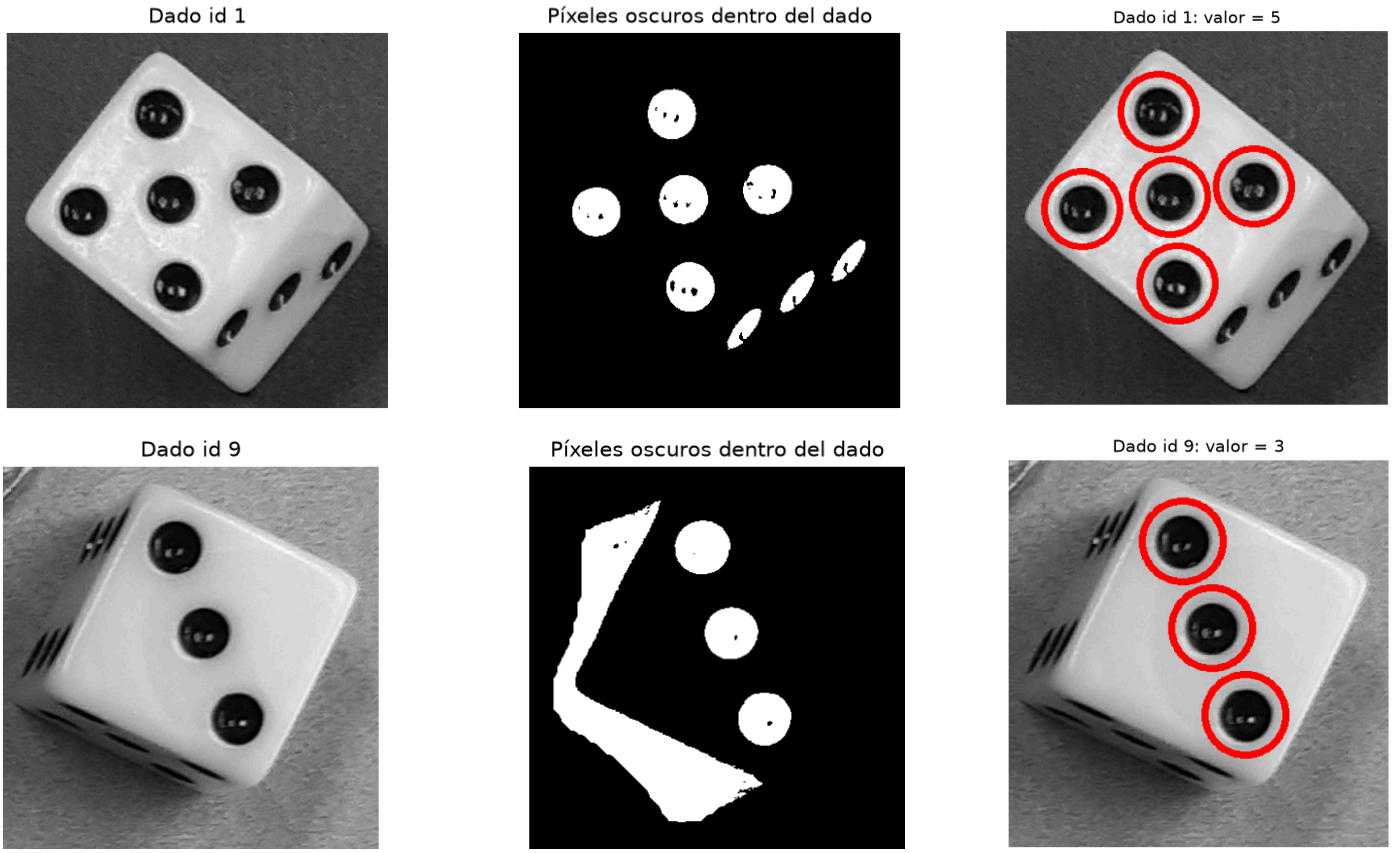


Figura 9. Ejemplo del procesamiento individual de cada dado: recorte original, detección de regiones oscuras y puntos identificados como válidos.

2.11 Resultado final

Una vez completadas las etapas de segmentación y clasificación se generó una imagen final que reúne los resultados. Cada objeto se representa con su contorno y una etiqueta: las monedas se identifican según su categoría de tamaño y denominación, y los dados muestran el valor detectado en su cara superior. El título de la imagen incluye el resumen del conteo.



2.12 Conclusión del Problema 1

La resolución de este problema requirió combinar distintas técnicas de procesamiento digital de imágenes para superar las dificultades de un fondo de intensidad no uniforme.

Durante las primeras pruebas se comprobó que el umbralado global mediante Otsu no permitía separar correctamente los objetos del fondo. Si bien el umbralado adaptativo mejoró visualmente la conservación de la información, la máscara obtenida presentó una fragmentación excesiva, que tampoco pudo resolverse satisfactoriamente mediante las primeras operaciones morfológicas probadas. Se optó entonces por el gradiente de Sobel sobre la imagen reducida, que no depende del brillo del fondo y a la vez eliminó el grano de la zona clara.

Con el umbral de bordes elegido por comparación, el filtrado por área y las operaciones morfológicas se construyó la máscara de los objetos. El punto clave fue completar cada objeto por separado: un cierre grande aplicado a toda la máscara une las monedas vecinas, mientras que sobre cada componente aislada permite cerrar los contornos sin perder la separación.

Sobre la máscara, la circularidad separó claramente dados y monedas, K-means agrupó las monedas en tres tamaños (9, 5 y 3 unidades) y el análisis de componentes conectados con filtros de área y circularidad permitió leer los puntos de la cara superior de los dados (5 y 3), descartando los de las caras laterales y la mancha oscura del dado derecho.

El procedimiento tiene limitaciones. Los parámetros (umbral de bordes, tamaños de los elementos estructurantes, umbral de circularidad y filtros de los puntos) se ajustaron a esta imagen y deberían revisarse para otra. Algunas monedas conservan pequeñas muescas en la máscara y el dado de la derecha queda con forma irregular, aunque ninguno de los dos efectos altera la clasificación. Además, la denominación de las monedas se infiere por su tamaño relativo.

En conjunto, el trabajo mostró la importancia de evaluar los resultados intermedios y de descartar con fundamento las alternativas que no funcionan, ajustando cada etapa a las características de la imagen.

3- Problema 2 – Detección automática de patentes

3.1. Introducción de las imágenes

Para este problema se contó con 12 imágenes (img01.png a img12.png) de la vista anterior o posterior de distintos vehículos. Todas tienen el mismo tamaño: 640 píxeles de ancho por 360 de alto, con 3 canales de color. Las patentes corresponden al modelo argentino de caracteres blancos sobre fondo negro, rodeados por un marco blanco, y están formadas por 3 letras y 3 números.

Las imágenes presentan condiciones distintas entre sí: cambian la iluminación, el color del vehículo y del entorno, la distancia a la cámara y la inclinación de la placa. Esto hace que un mismo procedimiento con parámetros fijos no se comporte igual en todas.

Al igual que en el problema anterior, cada imagen se cargó con cv2.imread() y se convirtió con cv2.cvtColor() a RGB, para visualizarla con Matplotlib, y a escala de grises, para procesarla. El procedimiento se desarrolló primero sobre una sola imagen (img01), mostrando el resultado de cada etapa, y luego se aplicó mediante un bucle a las 12 imágenes.



Figura 10. Imagen original (img01) e imagen en escala de grises utilizada para la detección de la patente.

3.2. Objetivo

El objetivo fue desarrollar un algoritmo capaz de encontrar automáticamente la patente en cada una de las imágenes. El procedimiento debía resolver dos tareas:

- **Detección de la placa:** localizar la patente dentro de la imagen y recortarla.
- **Segmentación de caracteres:** separar cada uno de los caracteres que componen la placa detectada.

En lugar de buscar directamente el rectángulo de la placa, cuyos bordes suelen confundirse con el paragolpes, la parrilla o la carrocería, se decidió buscar la patente a partir de sus caracteres. Una patente se reconoce por un patrón muy característico: seis objetos claros, de la misma altura, alineados en una fila y cercanos entre sí. Es muy poco probable que ese patrón aparezca por casualidad en otra parte de la imagen.

Para lograrlo se combinaron técnicas de umbralado, componentes conectadas, descriptores geométricos y recorte de regiones de interés.

3.3. Umbralado mediante Otsu

Como los caracteres son blancos sobre fondo negro, el primer paso fue binarizar la imagen para que cada carácter quedara como una región blanca separada. Antes de umbralar se analizó el histograma de la imagen en escala de grises.

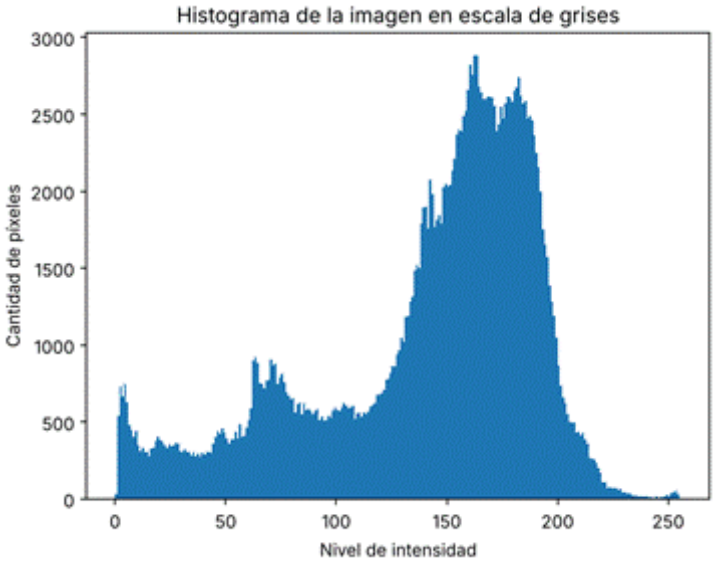


Figura 11. Histograma de la imagen en escala de grises (img01).

Se aplicó el umbralado global mediante el método de Otsu, que calcula automáticamente el umbral que mejor separa el histograma en dos clases. Para img01 el umbral obtenido fue T = 114. En la imagen binaria los caracteres de la patente quedaron blancos, pero también muchas otras regiones de la imagen, como el piso y partes del auto.

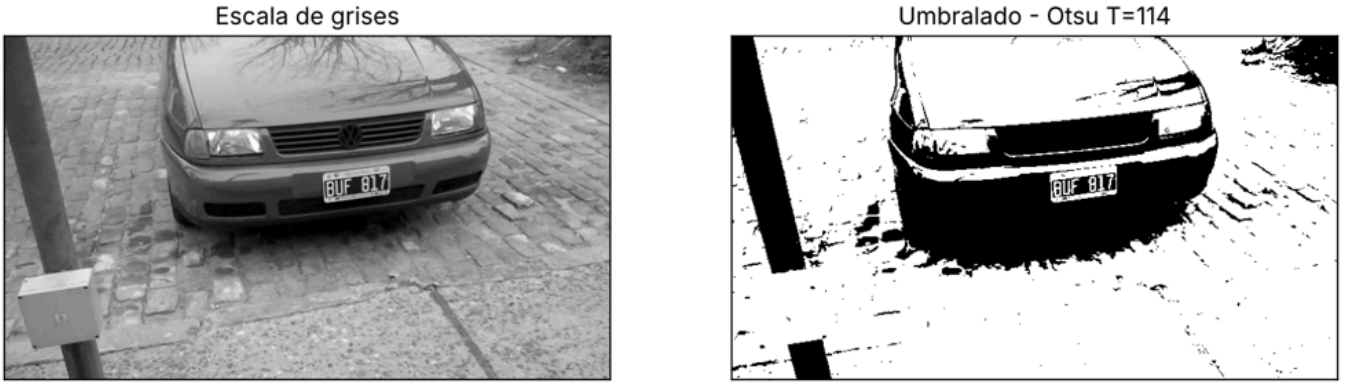


Figura 12. Imagen en escala de grises y resultado del umbralado mediante Otsu (T = 114).

3.4. Identificación de los objetos mediante componentes conectadas

Sobre la imagen binaria se utilizó cv2.connectedComponentsWithStats() para identificar las regiones independientes. Para cada una, la función devuelve su rectángulo delimitador (posición x, y, ancho w y alto h) y su área, que son los datos que se usaron en las etapas siguientes.

Se utilizó conectividad 4, es decir, solo se consideran conectados los píxeles vecinos de arriba, abajo, izquierda y derecha.

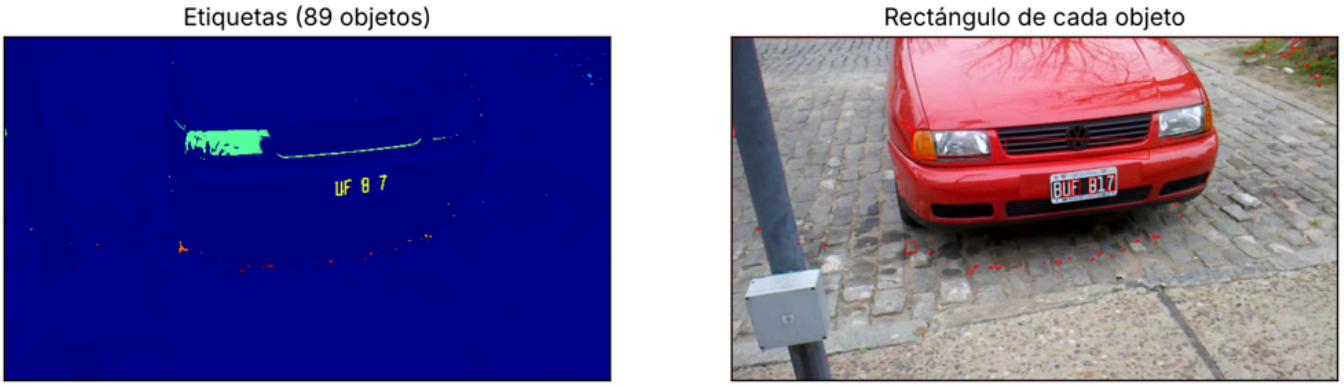


Figura 13. Etiquetas de las componentes conectadas y rectángulo delimitador de cada uno de los 89 objetos encontrados.

3.5. Filtrado de objetos por forma de letra

La mayoría de los 89 objetos no corresponden a caracteres. Para descartarlos se utilizaron dos descriptores geométricos obtenidos de los rectángulos delimitadores:

- **Alto entre 8 y 25 píxeles:** en estas imágenes los caracteres miden aproximadamente entre 10 y 20 píxeles de alto, y se dejó un margen hacia ambos lados. Se descartan el ruido, los objetos pequeños y los objetos grandes.
- **Relación de aspecto (alto / ancho) entre 1,2 y 3,5:** los caracteres son más altos que anchos. Se descartan las regiones cuadradas o más anchas que altas, como el marco o las rejillas del paragolpes.

El filtro es permisivo a propósito: no necesita ser perfecto, ya que los objetos que lo superan por casualidad se descartan en la etapa siguiente. En img01 quedaron 5 candidatos: cuatro caracteres de la patente y un borde del empedrado con forma de letra.



Figura 14. Todos los objetos encontrados y objetos que superaron el filtro por forma de letra.

3.6. Agrupamiento de las letras alineadas

Para quedarse solo con los caracteres de la patente se buscó el grupo de candidatos que respeta el patrón de una placa. Se tomó cada candidato como referencia y se lo comparó con todos los demás, considerando compatibles a los que cumplen tres condiciones:

- **Misma altura:** la diferencia de alto es de 4 píxeles como máximo.
- **Misma fila:** los centros verticales difieren como máximo en una altura de letra. Esta tolerancia permite reconocer patentes inclinadas.
- **Cercanía:** la distancia horizontal entre ambos es de 90 píxeles como máximo. La patente más ancha de las imágenes mide 78 píxeles de la primera a la última letra, por lo que se dejó un margen.

Se conservó el grupo cuya cantidad de integrantes estuviera más cerca de 6 y se lo ordenó de izquierda a derecha según la coordenada x. En img01 el borde del empedrado quedó descartado, porque no está alineado con ningún otro candidato, pero el grupo final tuvo solo 4 letras: Al hacer zoom sobre la patente se observó, además, que algunos caracteres quedaron unidos al marco blanco de la placa.



Figura 15. Candidatos y grupo de letras alineadas obtenido con el umbral de Otsu.

3.7. Selección del umbral de respaldo

Como el problema de img01 dependía del umbral, se probó el procedimiento con otros valores. Para poder repetir las etapas de las secciones 3.4 a 3.6 con distintos umbrales, se las agrupó en dos funciones: buscar_letras(), que obtiene las componentes conectadas y aplica el filtro por forma de letra, y agrupar_letras(), que devuelve el grupo de letras alineadas.

Se probaron umbrales por debajo y por encima del de Otsu: 85, 100, 114 (Otsu), 130, 145 y 160. Para cada uno se mostró la zona de la patente, ubicada alrededor de las letras ya encontradas con Otsu, con los caracteres detectados.



Figura 16. Letras encontradas en img01 con distintos umbrales.

Umbral	Letras	Resultado observado
85	1	La placa queda casi toda blanca y los caracteres se unen
100	1	La placa queda casi toda blanca y los caracteres se unen
114 (Otsu)	4	Algunos caracteres siguen unidos al marco
130	5	Todavía falta un carácter
145	6	Los caracteres se separan del marco (elegido)
160	6	También se encuentran los 6 caracteres

Bajar el umbral empeora el resultado, porque entra más blanco y todo se une. Subirlo, en cambio, separa los caracteres del marco, que es un poco más oscuro que ellos. Se eligió como umbral de respaldo T = 145, el menor valor con el que aparecen las 6 letras. Este valor no reemplaza a Otsu: solo se utiliza en las imágenes en las que Otsu no logra encontrar la patente.

3.8. Detección y recorte de la placa

Con las 6 letras encontradas se determinó la ubicación de la patente. Primero se calculó el rectángulo que contiene a todas las letras, tomando las coordenadas mínimas y máximas de sus rectángulos delimitadores. Luego se lo agrandó una altura de letra hacia cada lado, para incluir el marco completo de la placa.

Como el margen es proporcional al tamaño de las letras, funciona igual para vehículos cercanos y lejanos. Se verificó además que las coordenadas no quedaran fuera de la imagen. Finalmente se recortó la región de interés tanto de la imagen RGB, para visualizarla, como de la imagen en escala de grises, para procesarla en la etapa siguiente.



Figura 17. Patente detectada en img01 y región recortada.

3.9. Segmentación de los caracteres

Para separar los caracteres se umbralizó el recorte de la patente con el mismo umbral que había funcionado en la imagen completa. Dentro del recorte siguen apareciendo regiones que no son caracteres, como el marco, el texto «ARGENTINA» y los tornillos de la placa.

Se reutilizaron las mismas funciones de las etapas anteriores: el filtro por forma de letra descarta el marco y los elementos pequeños, y el agrupamiento conserva solo los caracteres alineados. De esta forma, el mismo criterio que permitió encontrar la patente en la imagen completa sirvió para segmentar sus caracteres. Finalmente, cada carácter se extrajo de la imagen binaria a partir de su rectángulo delimitador.

En img01 se segmentaron los 6 caracteres de la patente BUF 817, ordenados de izquierda a derecha.

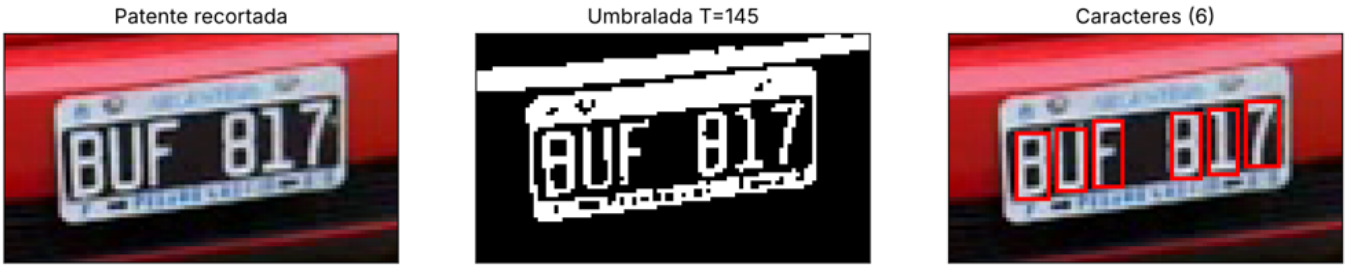


Figura 18. Patente recortada, recorte umbralado y caracteres segmentados.

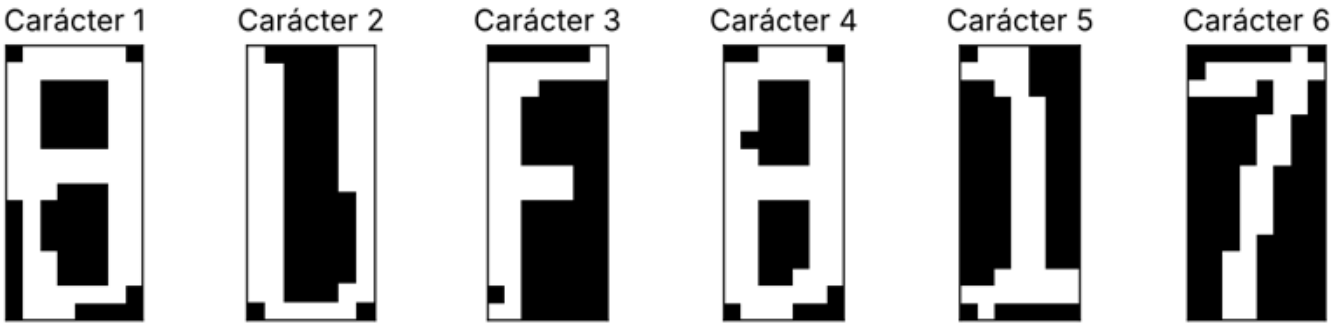


Figura 19. Caracteres segmentados de forma individual.

3.10. Procesamiento de las 12 imágenes

Una vez definido el procedimiento sobre img01, se lo aplicó mediante un bucle a las 12 imágenes. Para cada una se siguió la siguiente regla:

- Se aplica Otsu y se buscan las letras alineadas.
- Si no se encuentran 6 letras, se repite la búsqueda con el umbral de respaldo T = 145.
- Si con ninguno de los dos se encuentran 6 letras, la patente se informa como no detectada.

Con el umbral elegido se detectó y recortó la placa, y se segmentaron sus caracteres. Los resultados se informaron en forma de tabla:

Imagen	Otsu	Letras con Otsu	Letras con T = 145	Resultado
img01	114	4	6	Respaldo
img02	116	6	5	Otsu
img03	124	5	1	No detectada
img04	115	6	6	Otsu
img05	118	6	6	Otsu
img06	102	6	1	Otsu
img07	107	6	1	Otsu
img08	119	6	6	Otsu
img09	134	2	1	No detectada
img10	122	6	2	Otsu
img11	112	3	6	Respaldo
img12	92	6	2	Otsu

El umbral de respaldo es el mismo para las 12 imágenes; no se ajustó ningún umbral de forma particular para cada una.

Solo con Otsu se encontraron 8 de las 12 patentes, y con el umbral de respaldo se agregaron img01 e img11, llegando a **10 de 12**. En las 10 patentes detectadas se segmentaron los 6 caracteres. La tabla también muestra que el respaldo no puede reemplazar a Otsu: en img06, img07, img10 e img12, con T = 145 se encontrarían muy pocas letras.

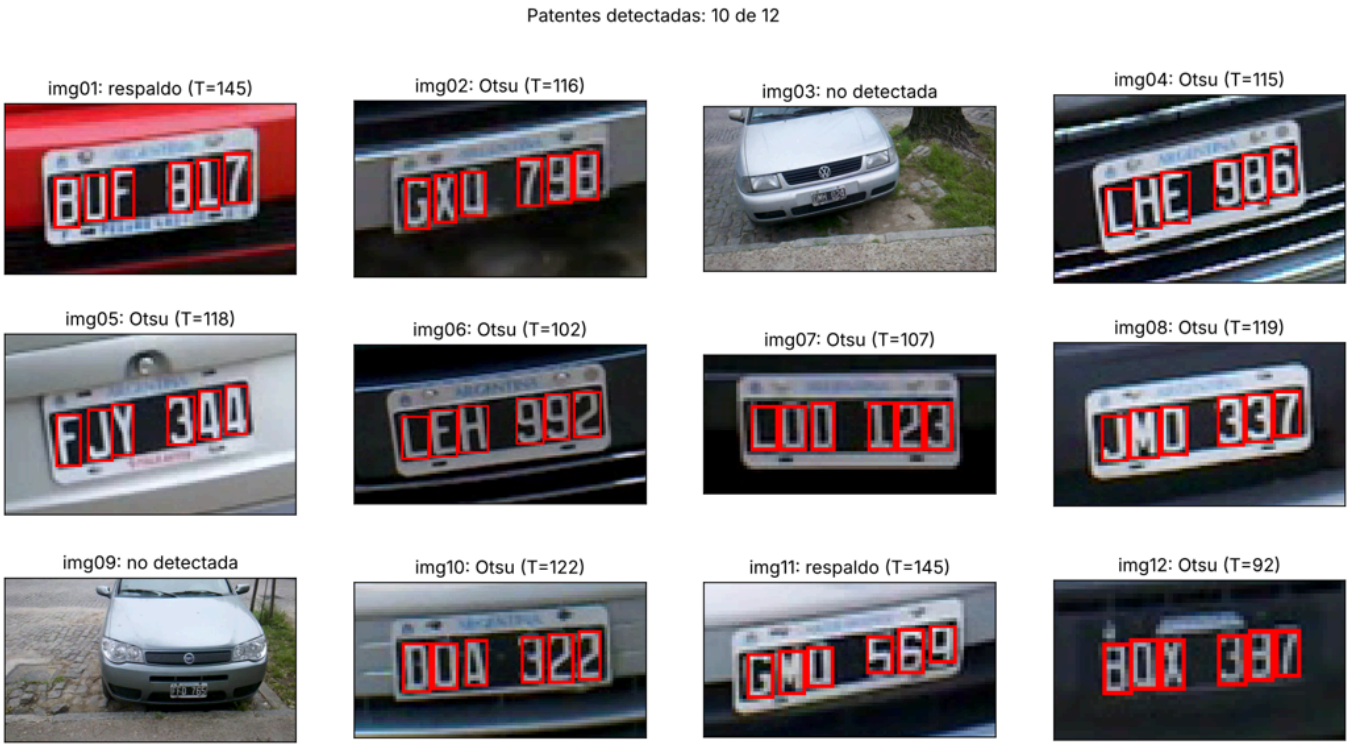


Figura 20. Resumen de las 12 imágenes: patentes recortadas con sus caracteres segmentados y umbral utilizado.

3.11. Análisis de las patentes no detectadas

Las dos patentes no detectadas (img03 e img09) fallan por el motivo opuesto al de img01 e img11. En estas últimas, Otsu entregaba un umbral bajo y el marco quedaba unido a los caracteres, por lo que subir el umbral resolvía el problema. En img03 e img09, en cambio, Otsu entrega un umbral alto (124 y 134) y los caracteres quedan finos y cortados en varios pedazos, que ya no tienen forma de letra. Subir el umbral a 145 empeora el resultado.

Esto se debe a que Otsu calcula el umbral con el histograma de toda la imagen y no solo de la patente. En img09 la fotografía es muy clara (un vehículo plateado sobre un piso claro), lo que desplaza el umbral hacia valores altos. En img03, además, la placa está inclinada y tiene poco contraste: con Otsu se encuentran 5 de las 6 letras y la que falta queda partida.

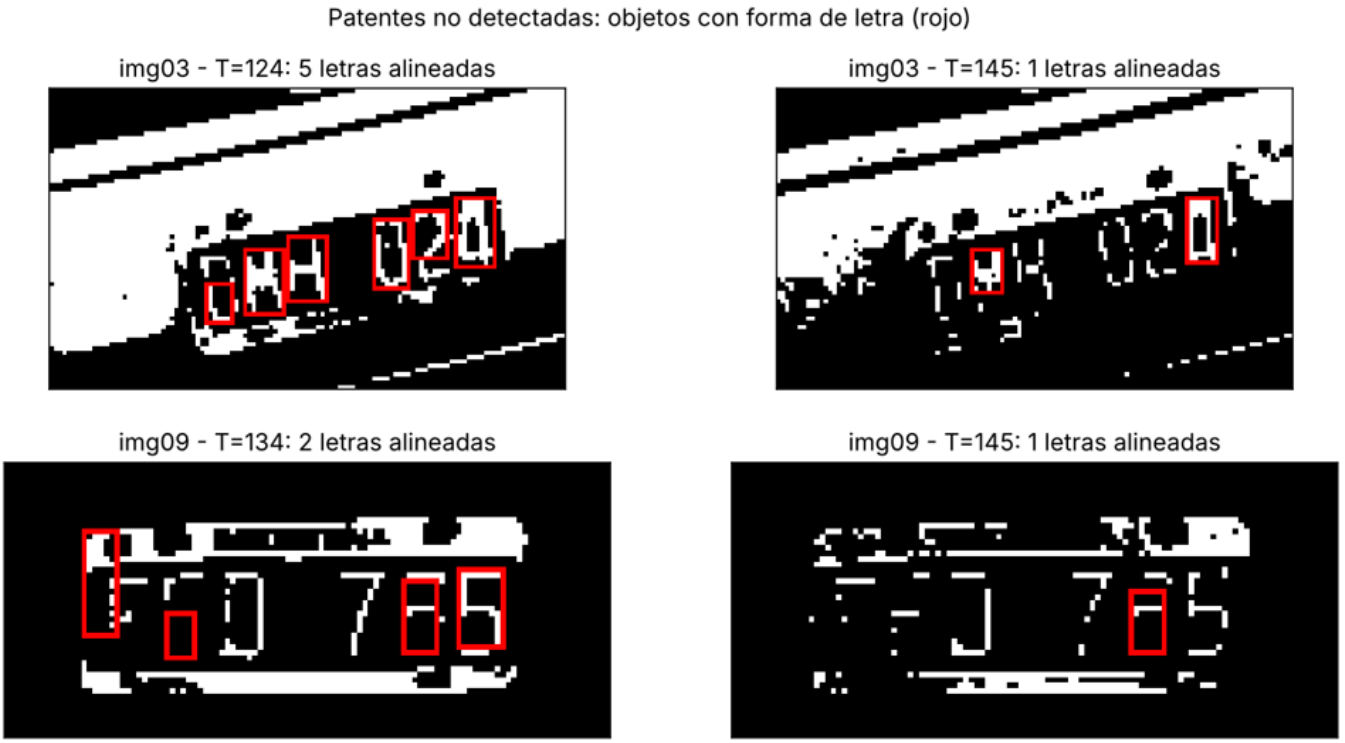


Figura 21. Zona de la patente en img03 e img09 umbralada con Otsu y con T = 145. En rojo, los objetos que superan el filtro por forma de letra.

3.12. Conclusión del Problema 2

La resolución de este problema se basó en una idea central: en lugar de buscar el rectángulo de la placa, se buscó el patrón que forman sus caracteres. Seis objetos de la misma altura, alineados y cercanos entre sí permitieron distinguir la patente del resto de la imagen, aun cuando el filtro por forma de letra dejaba pasar algunos objetos que no eran caracteres.

El umbralado de Otsu funcionó en 8 de las 12 imágenes. Su principal limitación es que calcula el umbral a partir de toda la imagen, por lo que en algunas fotografías no resulta adecuado para la patente. Para estos casos se incorporó un umbral de respaldo, elegido a partir del análisis de img01, que permitió llegar a 10 de 12 patentes detectadas. En todas ellas se segmentaron los 6 caracteres reutilizando las mismas funciones, lo que simplificó el procedimiento.

El procedimiento tiene limitaciones. Los parámetros del filtro (alto y relación de aspecto) están ajustados al tamaño de estas imágenes y deberían revisarse para otra resolución. El umbral de respaldo es único y solo resuelve los casos en los que hay que subir el umbral.

En conjunto, el trabajo mostró que combinar técnicas simples, como el umbralado, las componentes conectadas y los descriptores geométricos, con un criterio basado en la estructura del objeto buscado permite resolver la mayoría de los casos.